

# CH

## Hadoop - HDFS

- ☐ Big Data : Introduction
- ☐ Hadoop : présentation, cas d'utilisation, intégration
- ☐ HDFS
  - Architecture
  - Lecture/Ecriture
  - Rack Awareness
  - Fédération
  - Commandes
  - Exercice



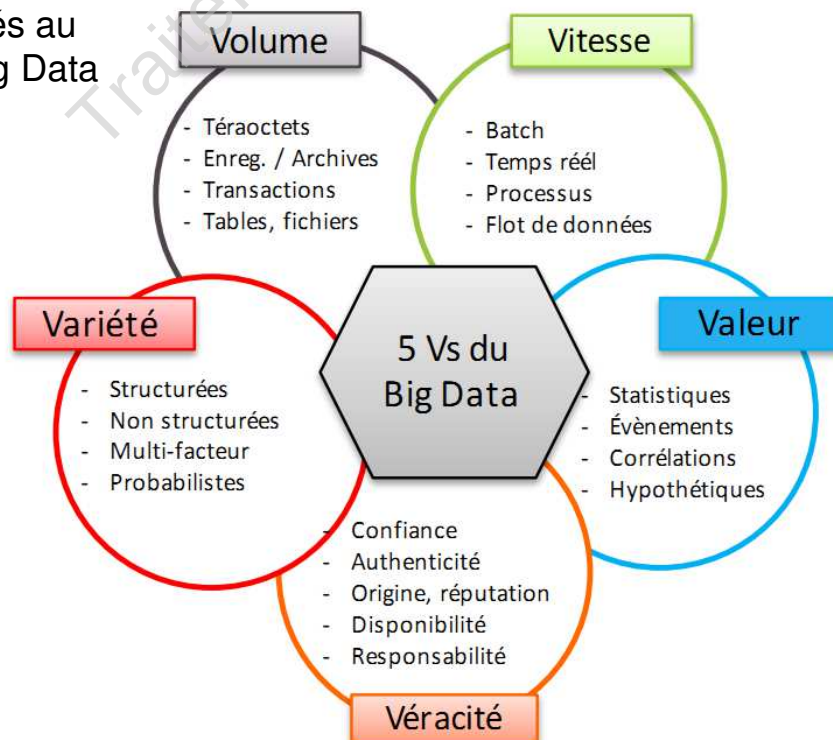
### Introduction



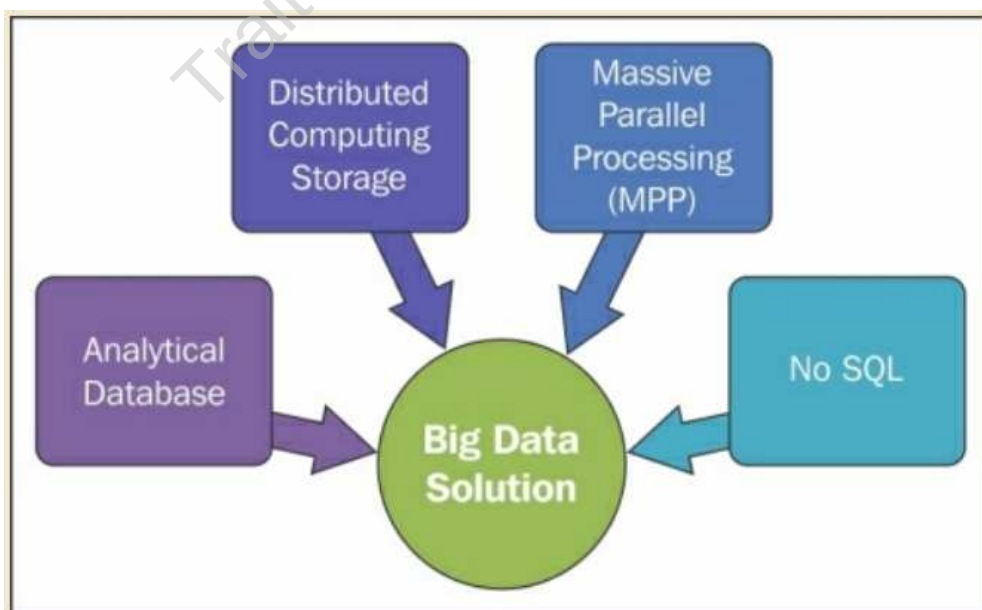
- Il y a une quantité énorme de données, disponibles sur Internet, dans les institutions, et avec certaines organisations, qui ont beaucoup des informations significatives, qui peuvent être analysées à l'aide de techniques de science des données et impliquent des algorithmes complexes.
- Les techniques de science des données exigent beaucoup de temps de traitement, de données intermédiaires et de puissance du processeur, ce qui peut prendre plusieurs heures. En outre, la science des données fonctionne sur une base d'essai et d'erreur, pour vérifier les performances des algorithmes en terme de traitement des données.
- Les systèmes Big Data peuvent traiter les données non seulement plus rapidement, mais aussi plus efficacement pour une grande quantité de données, que les systèmes classiques de traitement et d'analyse de données.

- Les systèmes Big Data ont émergé en raison de certains problèmes et de certaines limites dans les systèmes traditionnels.
  - Les systèmes traditionnels sont bons pour le traitement des (Online Transaction Processing (OLTP) et Business Intelligence (BI), mais ils ne sont pas facilement scalable (passage à l'échelle) compte tenu
    - \* coûts,
    - \* efforts
    - \* gestion
    - \* calculs lourds
    - \* Problème de mémoire
- Les systèmes traditionnels manquent énormément dans l'analyse des données scientifiques et rendent les systèmes de Big Data puissants et intéressants.
- Des exemples de cas d'utilisation de systèmes Big Data** sont l'analyse prédictive, l'analyse des fraudes, l'apprentissage automatique, l'identification des tendances, l'analyse des données, le traitement et l'analyse des données semi-structurées et non structurées.

Problèmes liés au traitement Big Data



- Les systèmes Big Data représente une terminologie qui fait référence aux défis qui surgissent en raison de la croissance exponentielle des données (Les 5V) :
  - Capture
  - Stockage
  - Recherche
  - Partage
  - Analyse
  - Visualisation
- Les technologies qui peuvent résoudre les problèmes de Big Data devraient utiliser la stratégie architecturale suivante :
  - Système d'informatique répartie
  - Traitement massivement parallèle (MPP)
  - Nosql (pas seulement SQL)
  - Base de données analytique



- Modèles de cas d'utilisation de Big Data : Il y a beaucoup de scénarios technologiques

- **Big data comme modèle de stockage**

- Les systèmes Big Data peuvent être utilisés comme modèle de stockage ou comme entrepôt de données, où les données provenant de sources multiples, même avec différents types de données, peuvent être stockées et peuvent être utilisées plus tard.

- **Big Data comme modèle de transformation des données**

- Les systèmes Big Data peuvent être conçus pour effectuer une transformation (chargement & nettoyage des données). Nombreuses transformations peuvent être effectuées plus rapidement que les systèmes traditionnels en raison du parallélisme.

- La transformation est une phase de du processus (**ETL : Extraction Transformation Loading**) (Extraction-Transformation-Ingestion)

- **Big Data pour un modèle d'analyse des données**

- L'analyse des données est d'un intérêt plus général pour les systèmes Big Data, où une énorme quantité de données peuvent être analysées pour produire des rapports statistiques et des aperçus sur les données

- **Big data pour des données en temps réel**

- Les systèmes Big Data intégrant certaines bibliothèques et systèmes qui sont capables de traiter des données en temps réel à grande échelle. Le traitement en temps réel pour un besoin important et complexe comporte de nombreux défis comme le rendement, l'évolutivité, la disponibilité, la gestion des ressources, la faible latence, etc.

- **Big data pour un modèle de cache à faible latence**

- Les systèmes Big Data peuvent être réglés comme un cas particulier pour un système à faible latence, où les lectures sont beaucoup plus élevées et les mises à jour sont faibles, ce qui peut récupérer les données plus rapidement et peut être stocké en mémoire, ce qui peut améliorer encore les performances.

## Introduction



### Visualization & Analytics



### Compute



### Storage



### Distributions & Data Warehouse



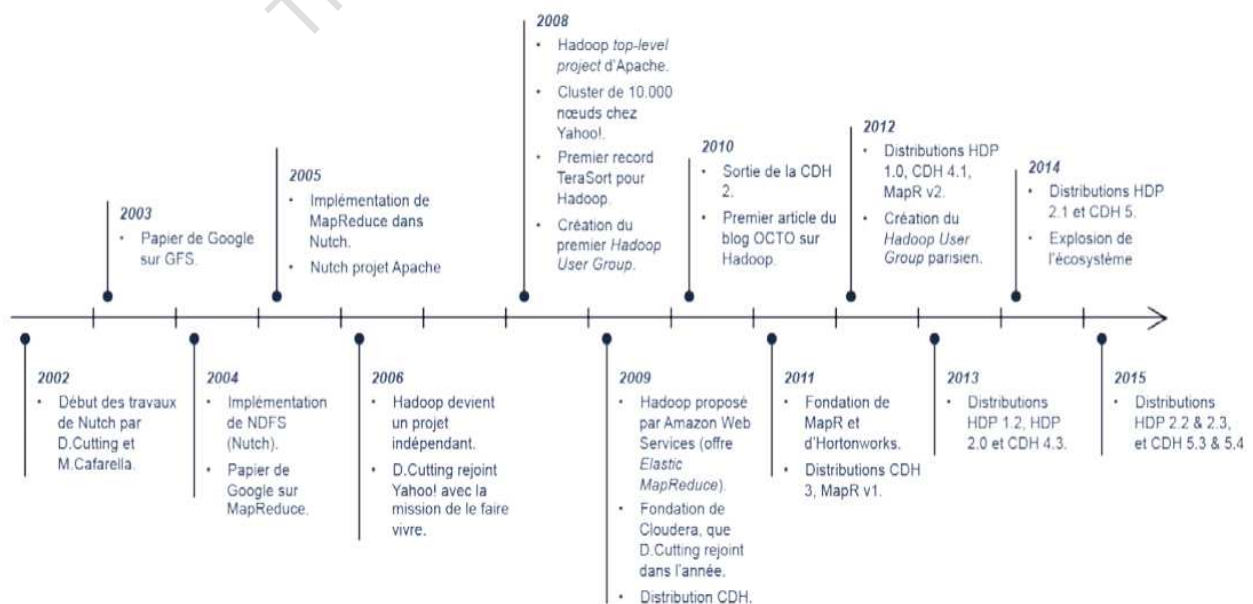
## Hadoop



- Dans le domaine de Big Data, le système le plus utilisé est Hadoop. Hadoop est une mise en œuvre open source de Big Data, qui est largement acceptée dans l'industrie.
- Les performances de Hadoop sont impressionnantes et, dans certains cas, incomparables à d'autres systèmes.
- Hadoop est utilisé dans l'industrie pour le traitement de données à grande échelle, massivement parallèle et distribué.
- Hadoop est très tolérant aux défaillances et configurable à plusieurs niveaux.
- Dans Hadoop, le calcul distribué est géré par HDFS, et le traitement parallèle est géré par Mapreduce.
- En bref, Hadoop est une combinaison de HDFS et Mapreduce.

- Hadoop a commencé à partir d'un projet appelé Nutch, une recherche open source sur les systèmes distribués.
- En 2003-2004, Google a publié les documents Google Mapreduce et GFS.
- Mapreduce a été adapté sur Nutch.
- Doug Cutting et Mike Cafarella sont les créateurs de Hadoop.
- Lorsque Doug Cutting s'est joint à Yahoo, un nouveau projet a été créé le long des lignes similaires de Nutch : appelé
- Plusieurs projets et sous projets ont commencé à s'intégrer à Hadoop, appelé écosystème Hadoop.

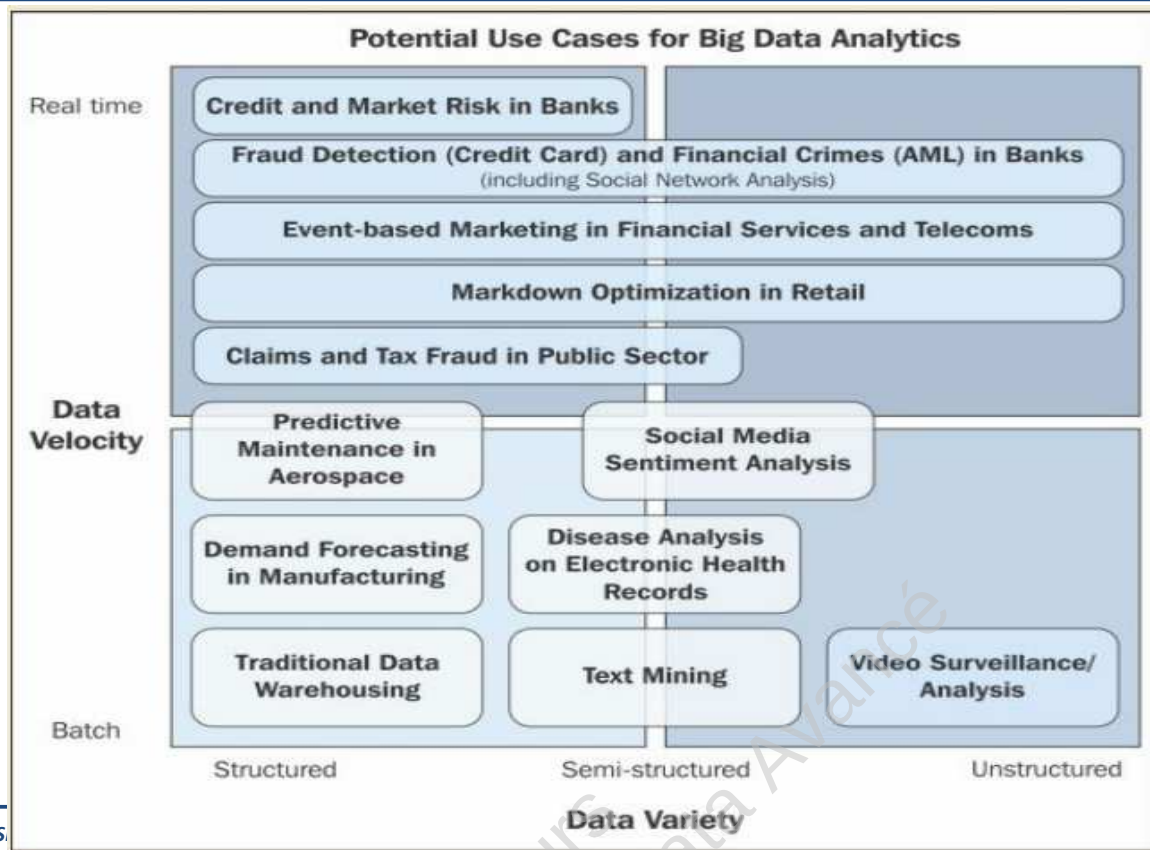
## Hadoop : les grandes dates clefs





- Faible coût – Fonctionne sur du matériel de base :  
Hadoop peut fonctionner sur du matériel de base (performant en moyenne) et ne nécessite pas un système de haute performance. Ce fait contrôle les coûts. Ajouter ou supprimer des nœuds du cluster est simple et à la demande. Le coût par téraoctet est plus faible pour le stockage et le traitement dans Hadoop.
- Flexibilité de stockage :  
Hadoop peut stocker des données au format brut dans un environnement distribué. Hadoop peut traiter les données non structurées et les données semi-structurées mieux que la plupart des technologies disponibles.
- Communauté open source :  
Hadoop est open source et soutenu par de nombreux contributeurs avec un réseau croissant de développeurs dans le monde entier. Soutenu par les géants de l'industrie des données (Yahoo, Facebook, Hortonworks, Cloudera, ...)

- Tolérance aux défauts :  
Hadoop est massivement évolutif et tolérant aux défauts. Hadoop est fiable en termes de disponibilité des données, et même si certains nœuds ne sont pas disponibles, Hadoop peut récupérer les données. L'architecture Hadoop suppose que les nœuds peuvent s'éteindre et que le système doit être en mesure de traiter les données.
- Analyse de données complexes :  
Avec l'émergence des Big Data, la science des données a également fait des bonds de géant, et des algorithmes complexes ont vu le jour. Hadoop peut traiter de tels algorithmes évolutifs pour des données à très grande échelle et peut traiter les algorithmes plus rapidement.



Un flux de données de base du système Hadoop peut être divisé en quatre phases :

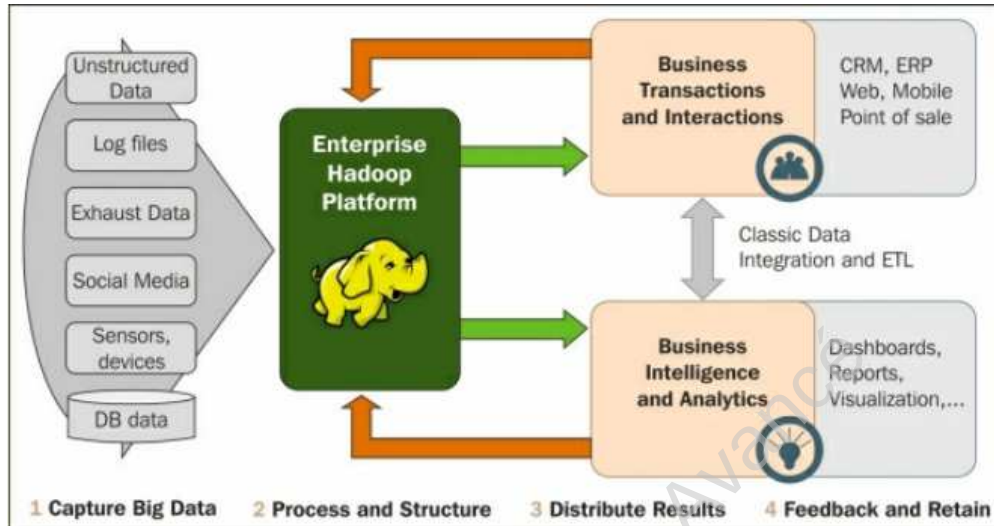
- Acquisition des données : Les sources peuvent être des listes exhaustives, structurées, semi-structurées et non structurées, certaines sources de données en continu, en temps réel, des capteurs, des appareils, des données saisies par machine et de nombreuses autres sources. Pour l'acquisition et le stockage des données, il existe différents intégrateurs de données comme Flume, Sqoop, Storm,... selon le type de données.
- Processus et structure : Nettoyage, filtrage et transformerons les données à l'aide d'un cadre basé sur Mapreduce ou d'autres cadres qui peuvent exécuter une programmation distribuée dans l'écosystème Hadoop (Mapreduce, Hive, Pig, Spark, ....)



## Hadoop : flue de données



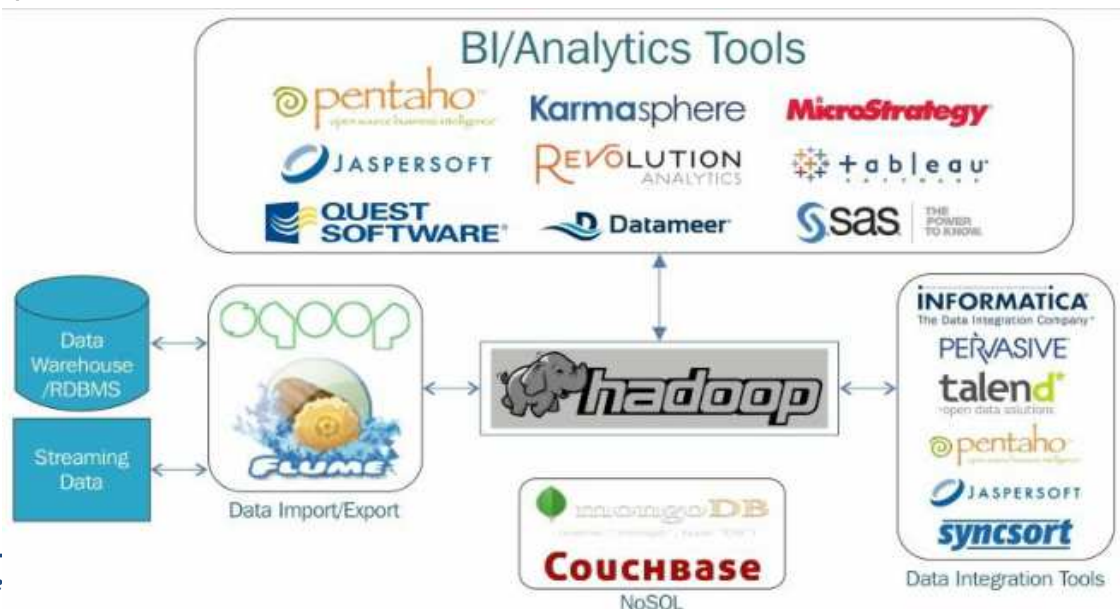
- Distribution des résultats : Les données traitées peuvent être utilisées par un système d'analyse ou de visualisations.
- Rétroaction et conservation : Les données analysées peuvent être transmises à Hadoop et utilisées pour des améliorations et des vérifications.



## Hadoop : Integration



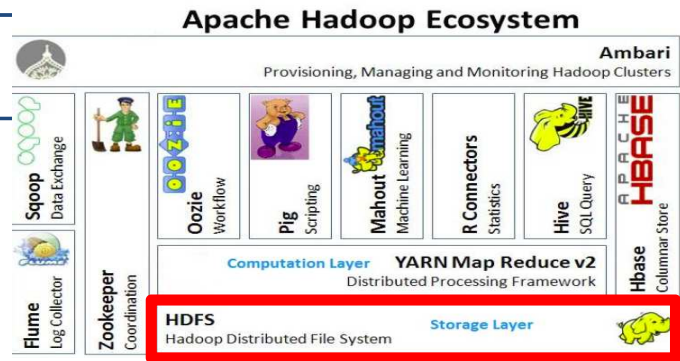
- L'architecture Hadoop est conçue pour être facilement intégrée à d'autres systèmes. L'intégration est très importante parce que même si nous pouvons traiter les données efficacement dans Hadoop, nous devrions également être en mesure d'envoyer ce résultat à un autre système pour déplacer les données à un autre niveau. Les données doivent être intégrées à d'autres systèmes pour assurer l'interopérabilité et la souplesse.



- Un cluster Hadoop peut être constitué de milliers de nœuds, et il est complexe et difficile à gérer manuellement, d'où certaines composantes qui facilitent la configuration, la maintenance et la gestion de l'ensemble du système Hadoop.

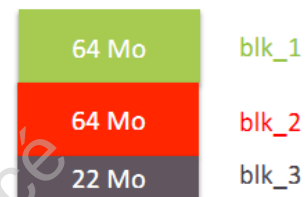
| L'ÉCOSYSTÈME HADOOP                                                                                    |                                    |                                 | Extraction, transformation, chargement / Outils « BI »                                                                   |                      |                                        |                          |
|--------------------------------------------------------------------------------------------------------|------------------------------------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------|----------------------|----------------------------------------|--------------------------|
| <b>Ambari</b> (Gestion « cluster », surveillance)<br><b>Hue</b> (Accès applications par interface web) | <b>Zookeeper</b><br>(Coordination) | <b>Oozie</b><br>(Planification) | <b>Impala</b>                                                                                                            | <b>Spark SQL</b>     | <b>Presto</b>                          | <i>Temps réel</i>        |
|                                                                                                        |                                    |                                 | <b>Pig</b><br>(Script)                                                                                                   | <b>Hive</b><br>(SQL) | <b>Sqoop</b><br>(Transfert de données) | <i>Accès aux données</i> |
|                                                                                                        |                                    |                                 | <b>HCatalog</b><br>(Meta Data)                                                                                           |                      |                                        | <i>Métadonnées</i>       |
|                                                                                                        |                                    |                                 | <b>HBase</b><br>(Stockage de type Colonne)                                                                               |                      |                                        | <i>Stockage table</i>    |
|                                                                                                        |                                    |                                 | <b>MapReduce</b><br>(Programmation distribuée)                                                                           |                      |                                        | <i>Distribution</i>      |
|                                                                                                        |                                    |                                 | <b>Yarn</b><br>(Gestion de ressources)                                                                                   |                      |                                        | <i>Ressources</i>        |
|                                                                                                        |                                    |                                 | <br>(Hadoop Distributed File System) |                      |                                        | <i>Stockage objet</i>    |

## HDFS : Architecture



- HDFS est un système de fichiers distribué, extensible et portable
- Ecrit en Java
- Permet de stocker de très gros volumes de données sur un grand nombre de machines (nœuds) équipées de disques durs banalisés à Cluster
- Quand un fichier mydata.txt est enregistré dans HDFS, il est décomposé en grands blocs (64Mo < Hadoop 2.2.0 & 128Mo dès Hadoop 2.2.0), chaque bloc ayant un nom unique: blk\_1, blk\_2...

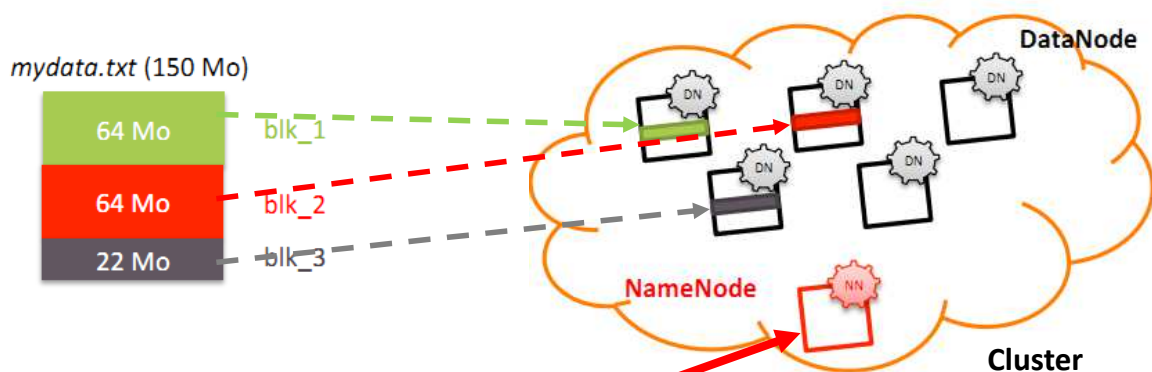
mydata.txt (150 Mo)



## HDFS : Architecture



- Chaque bloc est enregistré dans un nœud différent du cluster
- DataNode : démon sur chaque nœud du cluster
- NameNode : Démon s'exécutant sur une machine séparée
  - Contient des méta-données
  - Permet de retrouver les nœuds qui exécutent les blocs d'un fichier

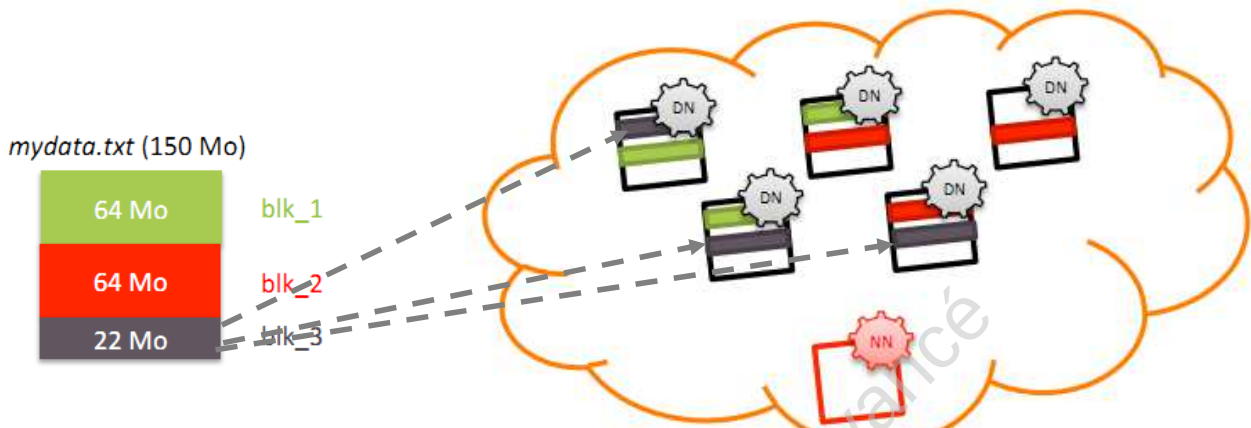


Méta-données (Les composants d'un fichier + emplacements)

## HDFS : Architecture



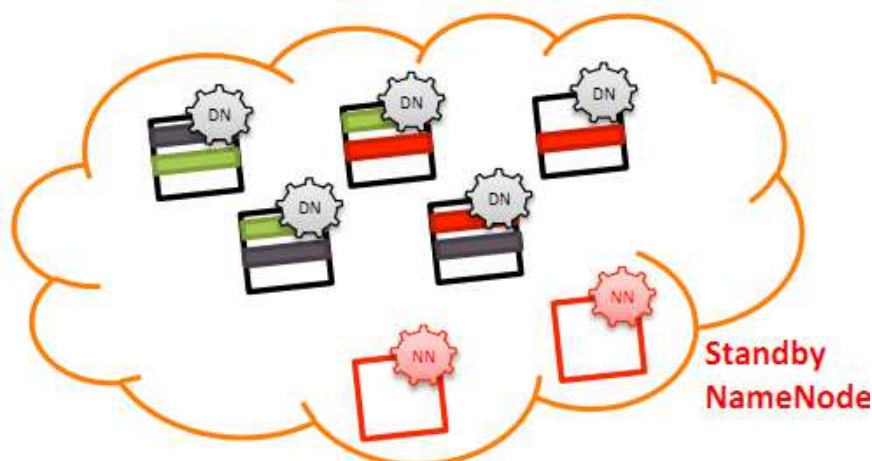
- Si l'un des nœuds a un problème, les données seront perdues
- ✓ Hadoop réplique chaque bloc 3 fois (par défaut)
- ✓ Il choisit 3 nœuds au hasard, et place une copie du bloc dans chacun d'eux
- ✓ Si le nœud est en panne, le NN le détecte, et s'occupe de répliquer encore les blocs qui y étaient hébergés pour avoir toujours 3 copies stockées
- ✓ Concept de Rack Awareness (rack = baie de stockage)



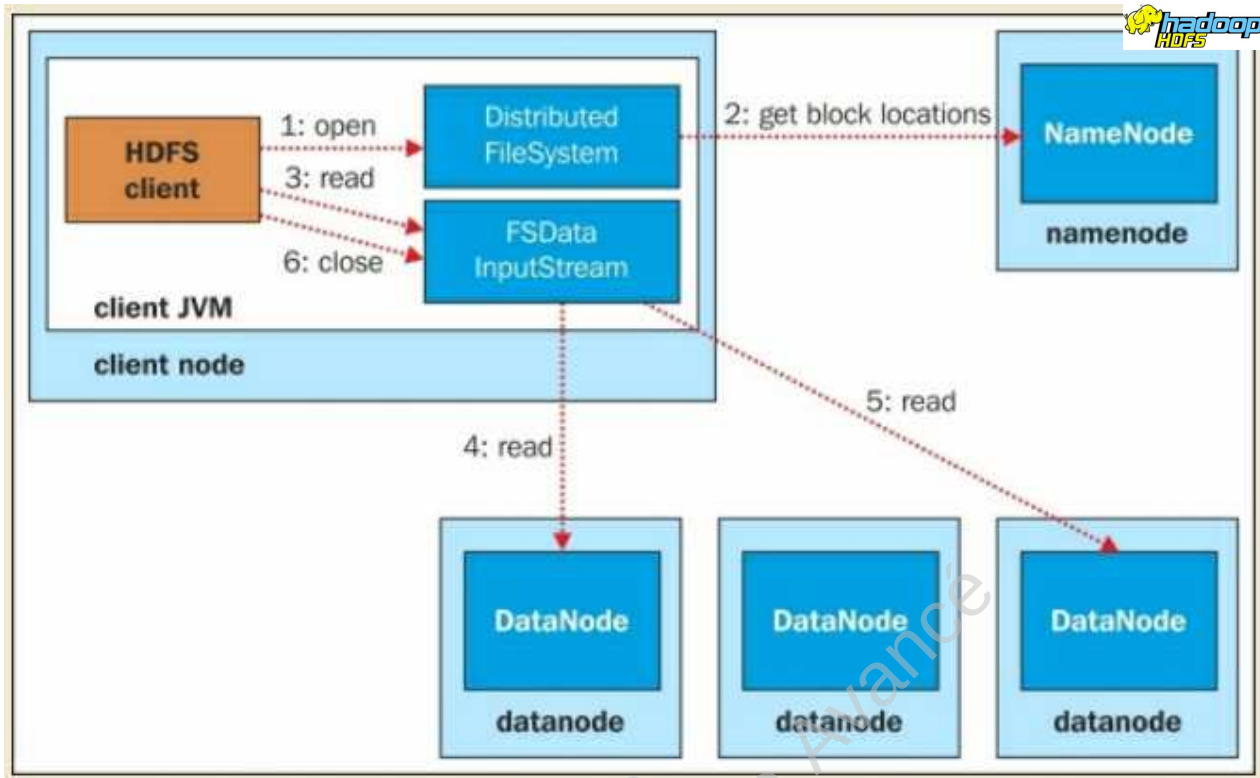
## HDFS : Architecture



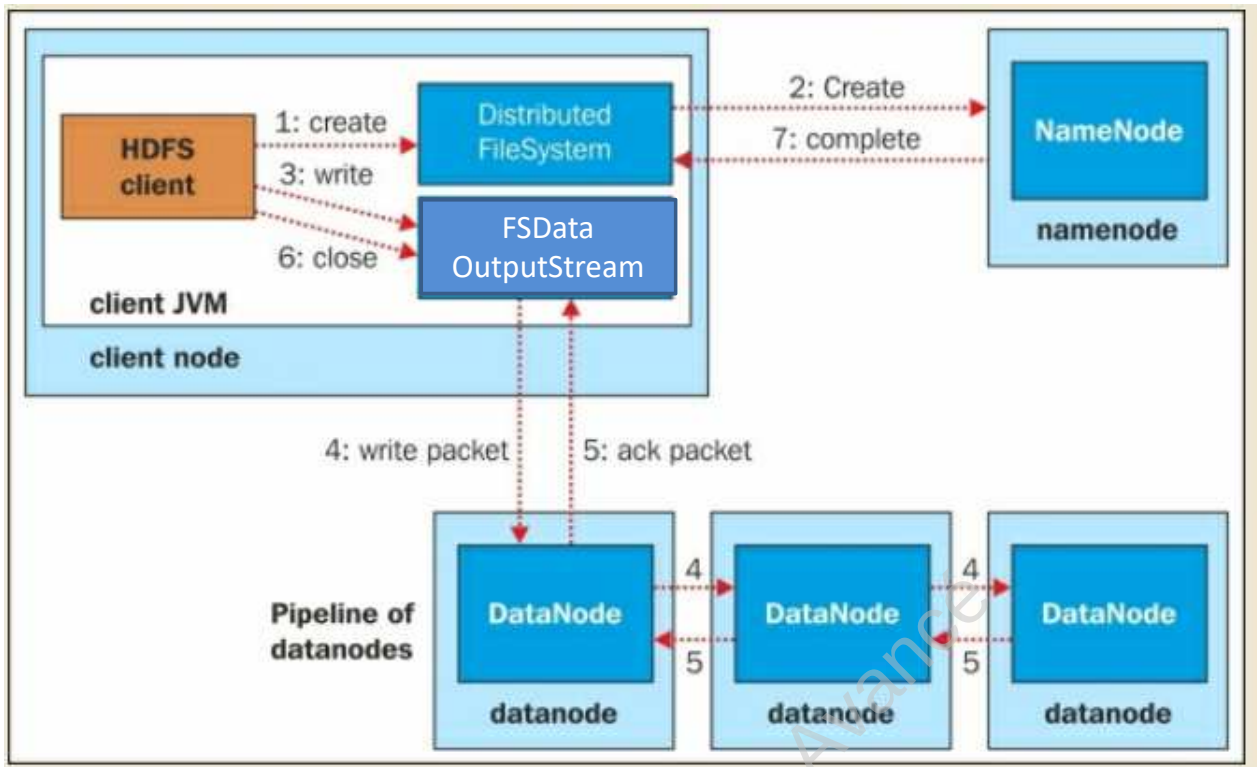
- Si le NameNode a un problème :
  - Données perdues à jamais
  - Données inaccessibles
- Définition d'un autre NN (standby namenode) pour reprendre le travail si le NameNode actif est défaillant







- 1) Le client utilisant un objet « Distributed FileSystem » de l'API client Hadoop appelle *open()* qui lance la requête read.
- 2) « Distributed FileSystem » se connecte avec « NameNode ». « NameNode » identifie les emplacements des blocs du fichier à lire et dans lesquels le bloc est situé. « Namenode » envoie ensuite la liste des « DataNodes » dans l'ordre des « DataNodes » les plus proches du client.
- 3) « Distributed FileSystem » crée ensuite des objets « FSDataInputStream », qui, à leur tour, enveloppent un « DFSInputStream », qui peut se connecter aux « DataNodes » sélectionnés et obtenir le bloc, et revenir au client. Le client initie le transfert en appelant la *read()* de « FSDataInputStream ».
- 4) « FSDataInputStream » appelle à plusieurs reprises la méthode *read()* pour obtenir les données du bloc.
- 5) Lorsque la fin du bloc est atteinte, « DFSInputStream » ferme la connexion à partir du « DataNode » et identifie le meilleur « DataNode » pour le bloc suivant.
- 6) Lorsque le client a terminé la lecture, il appellera *close()* sur « FSDataInputStream » pour fermer la connexion.

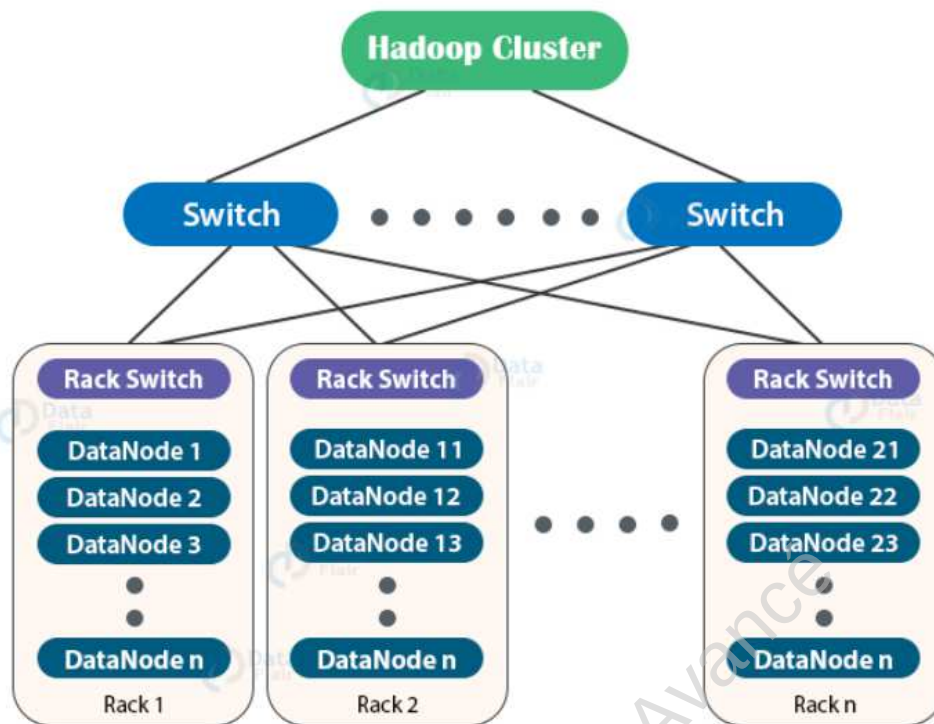


- 1) Le client, en utilisant un objet « Distributed FileSystem » de l'API client Hadoop, appelle *create()*, qui lance la demande d'écriture.
- 2) « Distributed FileSystem » se connecte avec « NameNode ». « NameNode » lance la création d'un nouveau fichier, crée un nouvel enregistrement dans les métadonnées et lance un flux de sortie de type « FSDDataOutputStream », qui enveloppe « DFSOutputStream » et le retourne au client. Avant de lancer la création du fichier, NameNode vérifie si un fichier existe déjà et si le client a des permissions pour créer un nouveau fichier et si l'une des conditions est vraie alors une « IOException » est envoyé au client.
- 3) Le client utilise l'objet « FSDDataOutputStream » pour écrire les données et appelle la méthode *write()*. L'objet « FSDDataOutputStream » gère la communication avec les « DataNodes » et « NameNode ».



- 4) « DFSOutputStream » divise les fichiers en blocs et les coordonne avec « NameNode » pour identifier les « DataNode » et les répliques de « DataNodes ». Les données seront envoyées à un « DataNode » dans des paquets, ce « DataNode » enverra le même paquet au deuxième « DataNode », le deuxième « DataNode » l'enverra au troisième, et ainsi de suite
- 5) Lorsque tous les paquets sont reçus et écrits, les « DataNodes » envoient un paquet d'accusé de réception au « DataNode » expéditeur et au client. « DFSOutputStream » maintient une file d'attente en interne pour vérifier si les paquets sont bien écrits par « DataNode ». « DFSOutputStream » gère également si l'accusé de réception n'est pas reçu ou si « DataNode » échoue pendant l'écriture.
- 6) Si tous les paquets ont été écrits avec succès, le client ferme le flux.
- 7) Si le processus est terminé, l'objet « Distributed Filesystem » notifie le « Namenode » de l'état.

- HDFS stocke des fichiers sur plusieurs nœuds (DataNodes) dans un cluster. Pour obtenir le maximum de performances de Hadoop et pour améliorer le trafic réseau pendant la lecture/écriture de fichiers, NameNode choisit les DataNodes sur le même rack ou les racks à proximité pour les données lues/écrites. « Rack awareness » est le concept de choisir le DataNode plus proche basé sur des informations rack.
- Le Rack est la collection d'environ 40-50 DataNodes connectés à l'aide du même commutateur réseau. Si le réseau tombe en panne, l'ensemble du rack ne sera pas disponible. Un grand cluster Hadoop est déployé dans plusieurs racks.

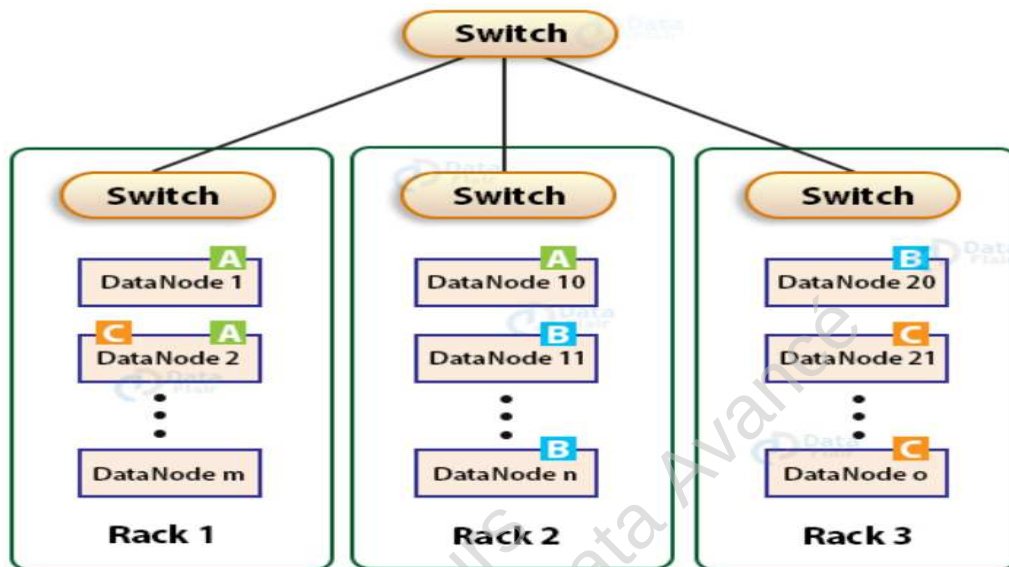


- Dans un grand cluster Hadoop, il y a plusieurs racks. La communication entre les DataNodes sur le même rack est plus efficace par rapport à la communication entre DataNodes résidant sur différents racks.
- Avantage de Rack Awareness
  - de Réduire le trafic réseau pendant la lecture/écriture du fichier, ce qui améliore les performances du cluster.
  - Pour atteindre la tolérance de défaut, même lorsque le rack n'est plus accessible
  - Atteindre une grande disponibilité des données même dans des conditions défavorables.
  - Pour réduire la latence (c'est-à-dire faire en sorte que le fichier lit/écrit avec un délai le plus faible possible).

## HDFS : Rack Awareness



- NameNode sur plusieurs rack cluster maintient la réplication bloc en utilisant des politiques de Rack Awareness suivante :
  - Pas plus d'une réplique soit placée sur un nœud.
  - Pas plus de deux répliques sont placées sur le même rack.
  - En outre, le nombre de racks utilisés pour la réplication de bloc devrait toujours être plus petit que le nombre de répliques.



FSB Bizer

I, 2 année

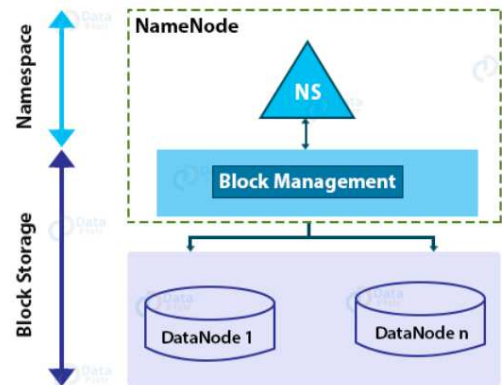
## HDFS : Fédération



- L'architecture de base HDFS prend en charge un seul NameNode qui gère l'espace de nom du système de fichiers.
  - La mémoire devient le facteur limitant pour l'échelle
  - Problème du goulot d'étranglement.
- La fonction de la Fédération HDFS ajoutée dans la version Hadoop 2.0 a permis à un cluster de s'étendre en ajoutant plus de NameNodes.

- Namespace

La couche Namespace de l'architecture **HDFS** se compose de fichiers, de blocs et d'annuaires. Cette couche fournit la prise en charge des opérations de système de fichiers liées à l'espace de nom comme créer, supprimer, modifier et répertorier des fichiers et des répertoires.



- La couche de stockage de bloc a deux parties :

- **Gestion des blocs** : NameNode effectue la gestion de bloc. Block Management fournit l'adhésion de DataNode au cluster en traitant les inscriptions et les battements périodiques. Il traite les rapports de bloc et prend en charge les opérations liées au bloc comme créer, supprimer, modifier ou obtenir l'emplacement du bloc. Il maintient également l'emplacement des blocs, le placement de réplique.
- **Stockage**: DataNode gère l'espace de stockage en stockant des blocs sur le système de fichiers local

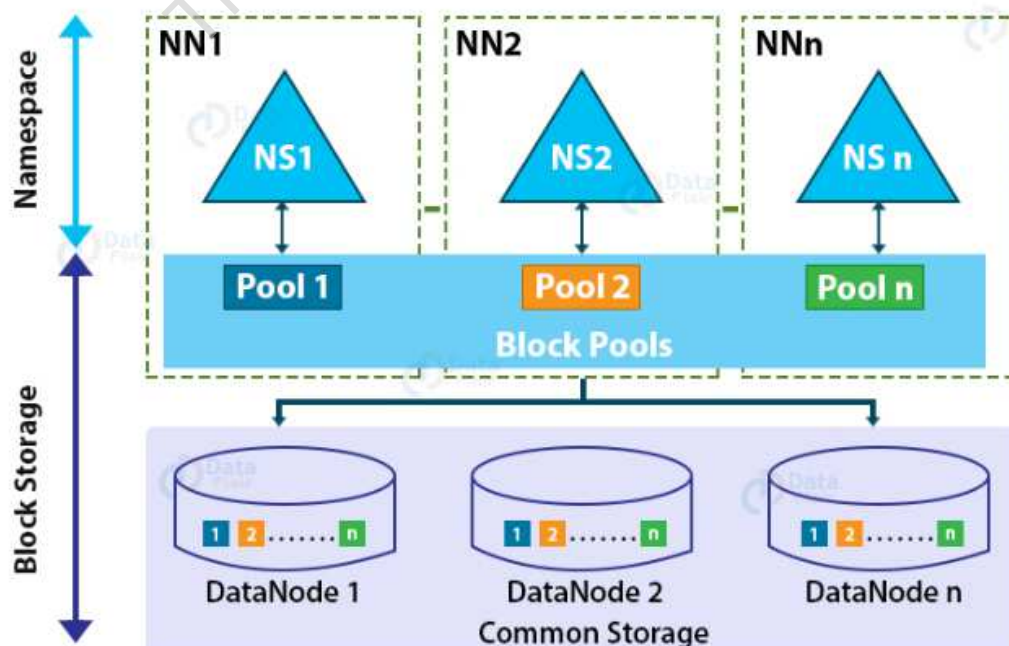
## HDFS : Fédération



- L'architecture de base ne permet **qu'un seul NameNode** pour maintenir l'espace nom du système de fichiers.
- Simple à implémenter et fonctionne bien pour les petits clusters.
- Pas adéquat pour les grandes organisations (Yahoo, Facebook,...)
- Limitations de l'architecture actuelle HDFS
  - Grâce à NameNode unique, nous ne pouvons avoir qu'un nombre limité de DataNodes qu'un seul NameNode peut gérer.
  - Les opérations du système de fichiers sont également limitées au nombre de tâches que NameNode gère à la fois. Ainsi, les performances du cluster dépendent du débit NameNode.
  - En raison d'un espace de nom unique, il n'y a pas d'isolement parmi les organisations d'occupants qui utilisent le cluster.

- La fonction HDFS Federation introduite dans Hadoop 2 améliore l'architecture HDFS existante.
- Il surmonte les limites d'architecture HDFS en ajoutant **plusieurs NameNode / namespaces**.
- Conséquent → scalabilité horizontale.
- Dans l'architecture de la Fédération HDFS, il existe plusieurs NameNodes et DataNodes.
- Chaque NameNode a son propre **espace de nom** et **block pool**.
- Tous les NameNodes utilisent tous les DataNodes comme stockage commun.
- Chaque NameNode est indépendant de l'autre (ne nécessite aucune coordination entre eux)
- Chaque DataNode est enregistré à tous les NameNodes dans le cluster
- DataNodes envoient périodiquement des battements de cœur à tous les NameNodes

### HDFS Federation Architecture





Commande en ligne  
API JAVA  
HTTP  
FTP

## HDFS : Commandes Shell



- Il existe 2 syntaxes possibles:
  - Avec **hadoop**: avec une syntaxe du type ***hadoop fs <commande>***
  - Avec **hdfs**: la syntaxe est ***hdfs dfs <commande>***.
- Cette commande sont proche de celles utilisées par le shell linux comme ls, mkdir, rm, cat
- Deux types de commandes
  - Linux Like :  
*cat, chgrp, chmod, chown, cp, du, ls, mkdir, mv, rm, stat, tail...*
  - Commandes Spécifiques  
*copyFromLocal, put, copyToLocal, get, getmerge, setrep...*
- Remarque  
La commande ***hadoop fs -help*** permet d'afficher la liste des commandes HDFS

## HDFS : Commandes Shell



Pour lister le contenu d'un répertoire

***hdfs dfs -ls <chemin du répertoire>***

Par exemple:

***hdfs dfs -ls /***

***hdfs dfs -ls /user # pour voir le contenu du répertoire "user"***

Pour afficher le contenu d'un fichier

***hdfs dfs -cat <chemin du fichier>***

Par exemple:

***hdfs dfs -cat /user/myFile.txt***

## HDFS : Commandes Shell



Pour créer un répertoire

***hdfs dfs -mkdir <chemin du nouveau répertoire>***

Par exemple:

***hdfs dfs -mkdir /user/output***



### Pour copier un fichier sur HDFS. on peut utiliser:

***hdfs dfs -put <chemin du fichier source> <chemin du fichier destination sur HDFS>***

La commande suivante est réservé seulement au fichier locaux:

***hdfs dfs -copyFromLocal <chemin du fichier sr> <chemin du fichier dest sur HDFS>***

Par exemple:

***hdfs dfs -put ~/TextFile.txt /user***

ou

***hdfs dfs -copyFromLocal ~/TextFile.txt /user***

### Pour récupérer un fichier de HDFS

***hdfs dfs -get <chemin du fichier sur HDFS> <chemin du fichier en local>***

Par exemple:

***hdfs dfs -get /user/TextFile2.txt ~/copyFromHadoop***

Cette syntaxe est réservée aux fichiers locaux:

***hdfs dfs -copyToLocal /user/TextFile2.txt ~/copyFromHadoop***

## HDFS : Commandes Shell

Pour vider la corbeille

***hadoop fs -expunge***



la commande **setrep** modifie le facteur de réplication à un compte spécifique au lieu du facteur de réplication par défaut. S'il est utilisé pour un répertoire, il changera de façon récursive le facteur de réplication de tous les fichiers résidant dans l'annuaire.

***hadoop fs -setrep 5 /user***

Pour Vérifier la taille du fichier

***hadoop fs -du /user/myFile.txt***

La commande **df** montre la capacité, la taille et l'espace libre disponibles sur le système de fichiers HDFS.

***hadoop fs -df***

## HDFS : Commandes Shell

Pour tester l'existence d'un fichier ou s'il est vide (1 si oui 0 si non)

**`hadoop fs -test /user/myFile.txt`**

Aide d'utilisation d'une commande particulière

**`hadoop fs -usage <command>`**

Pour changer les permissions d'un fichier

**`hadoop fs -chmod -r /user/myFile.txt`**

Extension d'un fichier

**`hadoop fs -appendToFile ~/file1 ~/file2 /user/appendFile`**

|           |                 |
|-----------|-----------------|
| touchz 01 | 07 chmod        |
| test 02   | 08 appendToFile |
| text 03   | 09 checksum     |
| stat 04   | 10 count        |
| usage 05  | 11 find         |
| help 06   | 12 getmerge     |

## HDFS : API JAVA



- Hadoop propose une API Java complète pour accéder aux fichiers de HDFS. Elle repose sur deux classes principales :
  - **FileSystem** représente l'arbre des fichiers (file system). Cette classe permet de copier des fichiers locaux vers HDFS (et inversement), renommer, créer et supprimer des fichiers et des dossiers
  - **FileStatus** gère les informations d'un fichier ou dossier :
    - taille avec `getLen()` ,
    - nature avec `isDirectory()` et `isFile()` ,
- Ces deux classes ont besoin de connaître la configuration du cluster HDFS, à l'aide de la classe **Configuration**. D'autre part, les noms complets des fichiers sont représentés par la classe **Path**

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.FileStatus;
import org.apache.hadoop.fs.Path;
import java.io.*;

public class HDFSread {
    public static void main (String[] args) throws IOException {
        Configuration conf = new Configuration();
        FileSystem fs = FileSystem.get(conf);
        Path nomcomplet = new Path( "myFile.txt" );
        FSDataInputStream inStream = fs.open(nomcomplet);
        InputStreamReader isr = new InputStreamReader(inStream);
        BufferedReader br = new BufferedReader(isr);
        String line = br.readLine();
        System.out.println(line);
        inStream.close();
        fs.close ();
    }
}
```

## Exercice



- 1) Créer un repertoire "**rep1**" Hadoop HDFS directement sur la racine
- 2) Vous avez un fichier "**myFile.txt**" en local **~/MPDS/Level1**. Copier le fichier myFile sur le repertoire "**rep1**" par deux methods
- 3) On vous demande de déplacer le fichier "myFile.txt" vers un deuxième repertoire "**rep2**"
- 4) Afficher l'entête et l'en-que du fichier myFile depuis "rep2"
- 5) Vérifier sa taille
- 6) Faites de sorte que le fichier soit dupliqué 4 fois sur le cluster Hadoop. En tenant compte de la federation de Hadop, faites un schema explicatif montrant le stockage du fichier sur le cluster
- 7) Tester si le fichier myFile existe encore dans rep1